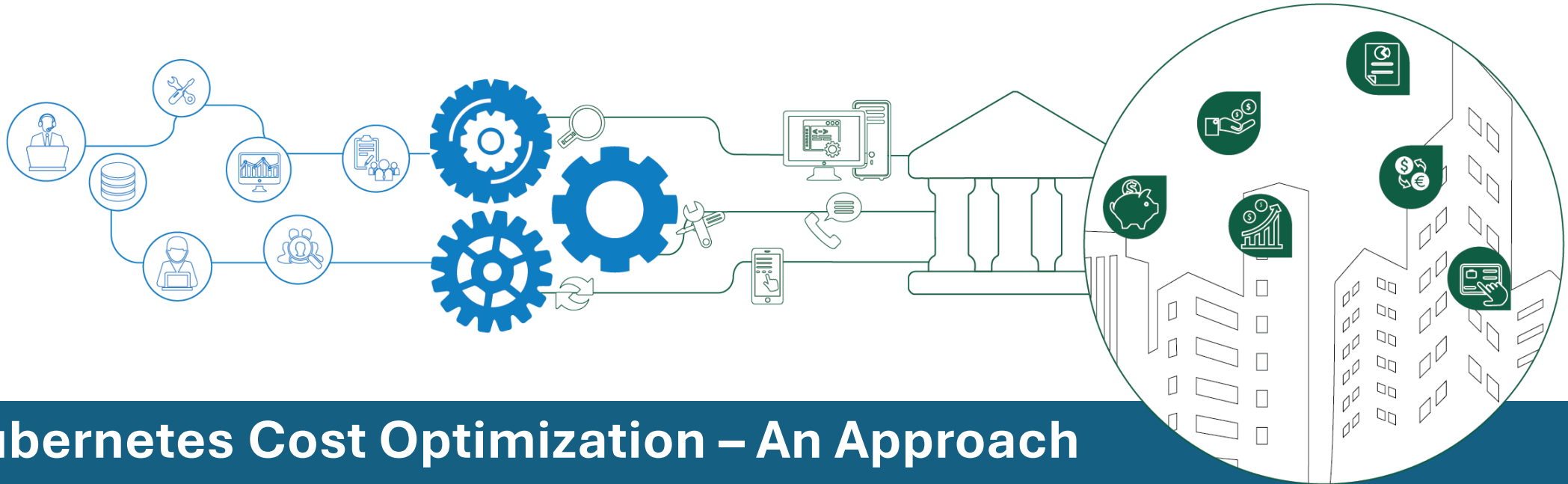




Agenda

- Current Challenges
- Capacity Planning and Resource Optimization
- Resource Cost Optimization
- Case Study
- Network Cost Optimization
- Storage Cost Optimization
- QAs

Current Challenges



- Not using the right nodes(instance types) of the right use cases. Over provisioning or underusing Cpu/Memory.
- Challenges with number of pods running on nodes.
- Challenges with resource constraints in pods.
- Node Auto provisioning without proper capacity planning.
- Resource Governance
- Not using tags and labels properly.
- SLA Breaches and Developer Challenges
- Effective use of Autoscaling parameters.

Kubernetes Cloud Costs

Fixed Cost	Workload Cost	Shared Cost	Optional/External Cost
Control Panel	Containers: CPU Memory	Network	Container Registries
	Storage	LoadBalancers	Disaster Recovery
		NAT	Storage
		Observability	



Resource Optimization/Cost Optimization - Checklist

If you are looking at a migration project where your existing workloads are running in other Kubernetes platforms like OpenShift or your workloads are hosted as VMs in on premises, then please do the following.

Volumetric data - This includes the Number of requests/loads on production(Min/Average/Max) and Latency details. Collect the expected and predicted load in the future say for next 6 months or 1 year. This will also help you setting up scaling parameters.

Understand the current landscape - Number of front-end application/services, backend services, databases, dependent systems and third-party products and services.

Capture current infrastructure details – Cpu/Memory/Network and Storage with VM Flavours

Resource Optimization/Cost Optimization - Planning

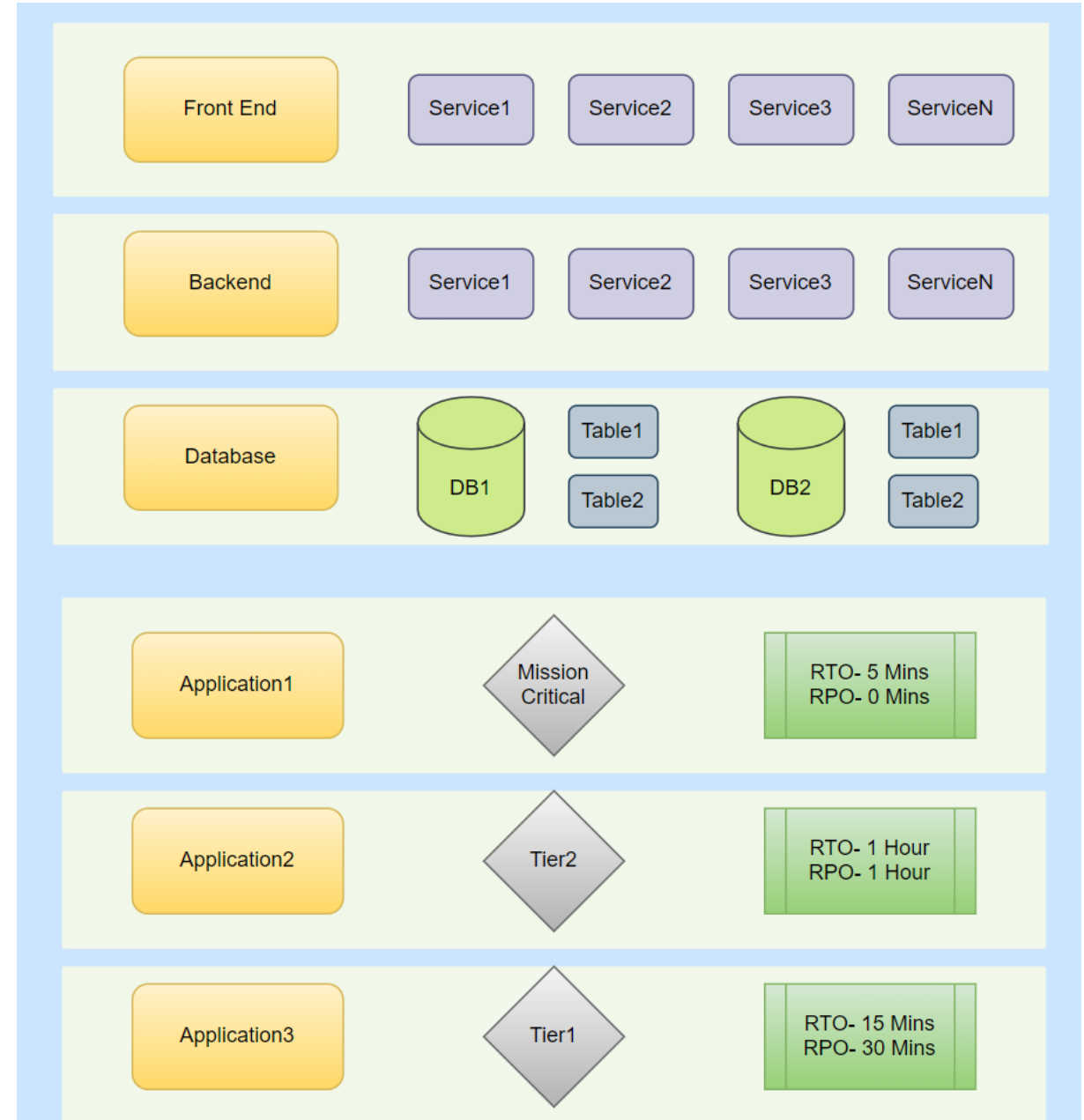
Resource Classification

See the diagram, the number of resources for each layer helps you to plan and define how many clusters you may need and how many nodes you need to create. We will talk more about this in the later part of this document.

Identify Your Application Tier

This is your first step for capacity planning. Identifying and categorizing your application tier is mainly used for your SLA Requirements. Some of your services might be mission critical and need high availability requirement of 99.99 and some might not be much critical where the availability might be little low.

This categorization helps you plan your capacity, scaling and HA-DR requirements and ensures your SLA is not breached.



Resource Optimization/Cost Optimization - Planning

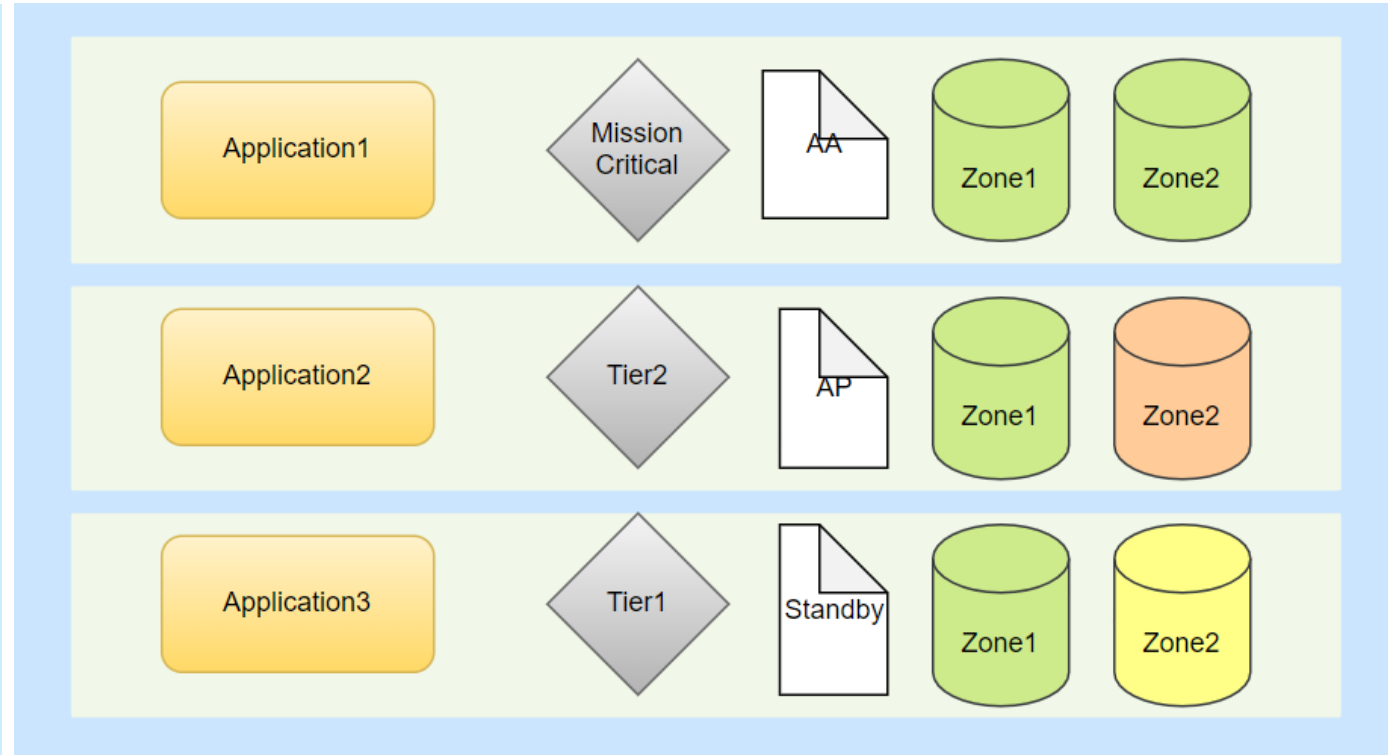
Identify your Geography:

AA- Active Active Setup, AP- Active Passive Setup,
Standby- Warm Standby

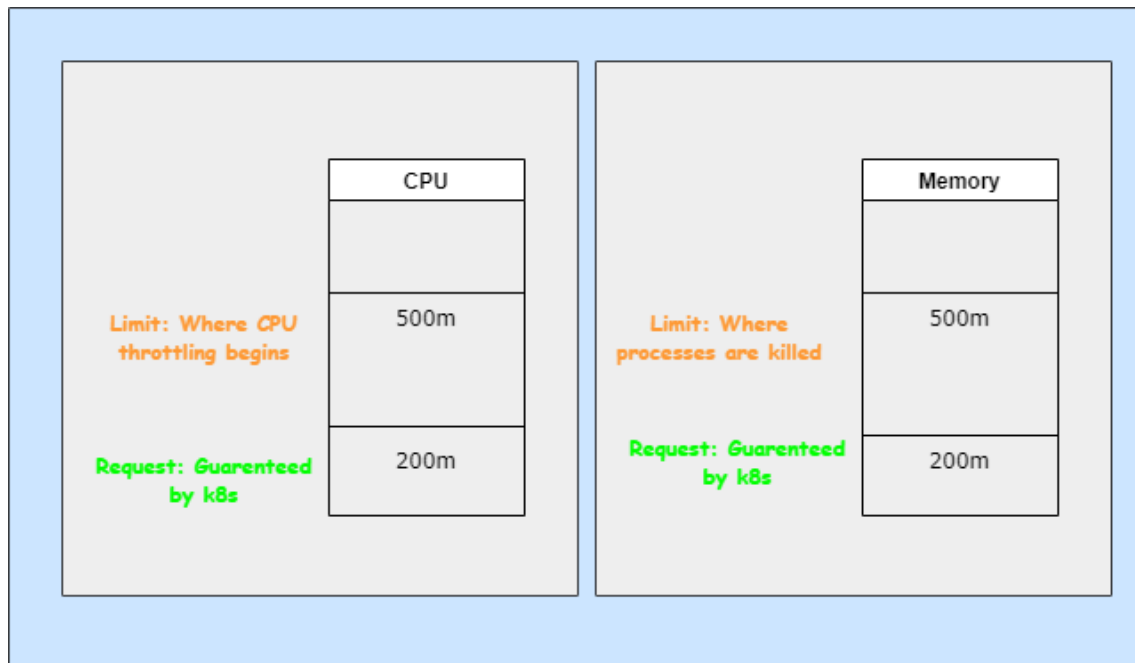
Choosing Right Instance Types:

Your instance type will be judged/selected based on the following parameters:-

1. Type of environment. For dev and lower environments go for a smaller instance types.
2. HA and DR – Cost of instances may vary from region to region. Please plan accordingly.
3. NFRs like latency, throughput, real time events.
4. Type of Services. For example, Databases - Storage Optimizes instances, Cache Services – Memory Optimized instances etc.



Resource Optimization/Cost Optimization – Request and Limits



Resources	Low	High
Request CPU	Starvation, Might not get cpu cycle	Inefficiency, Other pods will not be scheduled as the resources are on hold by single resource
Request Memory	High Risk to be killed, Resource will be pulled by other pods	Inefficiency, Other pods will not be scheduled as the resources are on hold by single resource
Limit CPU	CPU Throttling	Inefficiency, other pods starve
Limit Memory	Pod will be killed by kernel	Inefficiency, other pods starve

Resource Optimization/Cost Optimization – Request and Limits

This best practice of analyzing historical usage patterns typically provides both a good representation of the future and is easy to understand/introspect. Applying extra headroom allows for fluctuations that may have been missed by your historical sampling. We recommend measuring usage over 1 week at a minimum and setting thresholds based on the specific availability requirements of your pod.

Tier	Request	Limit
Critical / Highly Available	99.99th percentile + 100% headroom	2x request or as higher if resources available
Production / Non-critical	99th + 50% headroom	2x request
Dev / Experimental	95th or consider namespace quotas*	1.5x request or consider namespace quotas*

Lets see with an example

Service	Desciption	Tier	SLA	Requests Per Minute	Request CPU	Limit CPU
Service A	Payments	Mission Critical	99.99	1000	1vCPU	2vCPU
Service B	Accounts	Non Critical	99.5	200	0.2 vCPU	0.5vCPU

Resource Quota and Limits at Namespace Level

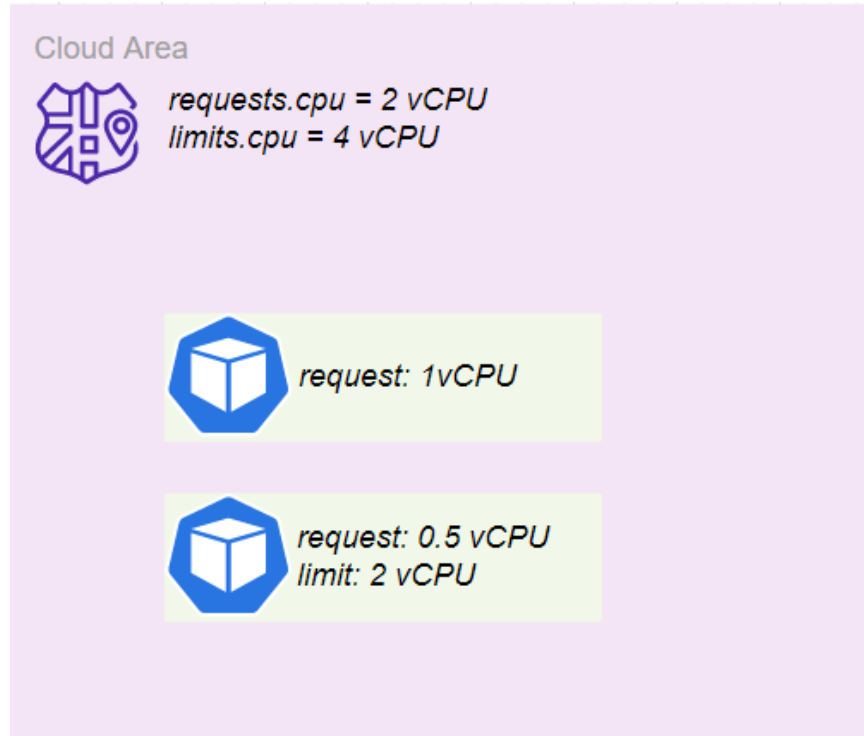


Figure1 : A Simple scenario, How request and limits at pod level works with settings at Namespace Level

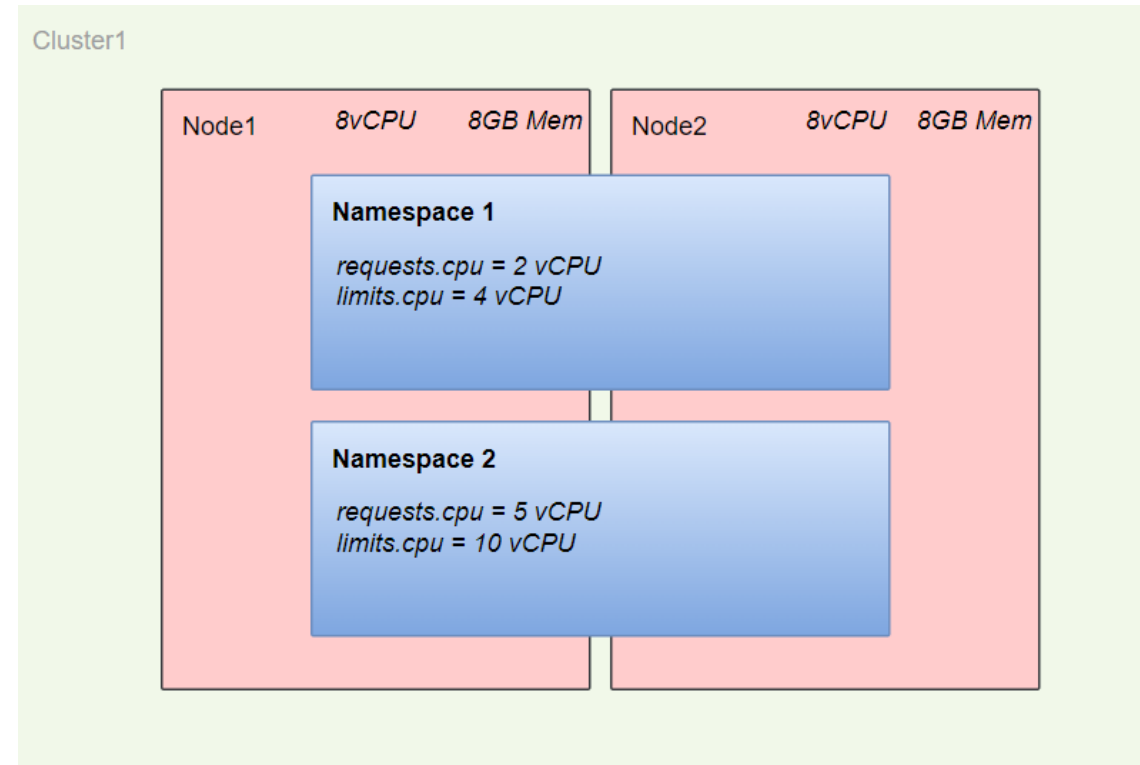


Figure2 : How your requests and limits set at namespace level works at cluster level

Performance Testing

Key Performance Test designs to address above objectives are

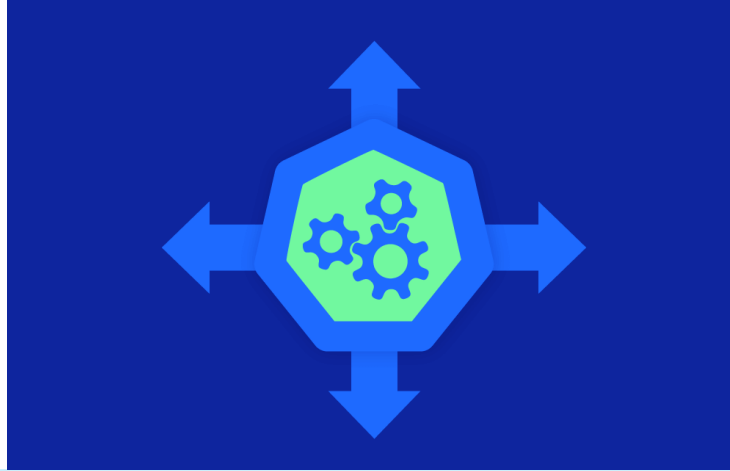
- *Avg and Peak Load Tests*
- *Endurance Tests*
- *Max Load Tests (currently we are testing 1.5 times the current expected)*
- *Test early in the life cycle i.e. conducting shift left tests in lower environment like SIT/SAT and have some baseline and raise defects early in the testing phases*

Standard Metrics that are monitored across any Performance Test using the availability tool set: JMeter/LoadRunner, Dynatrace, CloudWatch, ELK, Cloud Logging in GCP

Capacity Planning tests can be achieved by validating the Max Load Tests – (We may need to have a dedicated environment for this for flexibility). In some application, the stress test can go up to 3X – 4X as required.

Results (vCPU, Mem)					
1.5, 2.5	1, 2	0.75,1.5	0.5,1	0.25,.512	Remarks
Latency @95th Percentile in Milliseconds					
1989	2502	6562	22630	39981	NA
1692	7897	6380	13336	24817	NA
4564	15970	11469	26951	51159	NA
1248	6778	2638	5475	10307	NA
1722	9607	2604	6489	10136	NA
65	192	284	872	2326	NA
UAT1	UAT1	UAT1	UAT1	UAT1	NA

Optimizing Cluster Autoscaling



- Reduce ***scale-down-unnneeded-time*** – This setting controls how long a node must be underutilized before it is removed. Default value is 10 mins and you can reduce this to scale down faster if required.
- Use ***priority expander*** – The priority expander allows you to define which node group should be prioritized during scale-up. Example scenario, Prioritize spot instances over on demand.
- ***Scale-down-utilization-threshold***
- ***Scale-down-delay-after-add***
- ***Max-node-provision-time***

Client Case Study

Client Profile

- American insurance and financial services company headquartered in Pennsylvania. This Financial Organization provides high-quality investments, personalized financial advice, retirement tools and relevant market insights.

Tech Stack

- Java Microservices, Angular, GraphQL, ELK, AWS, EKS, Postgres Aurora.

Project Challenges

- No Uniformity on Pods.
- Resource Constrains are not properly setup.
- No proper Labels or Observability enabled.
- Resource governance was not established properly.
- Autoscaling strategy was not properly used.
- Overutilization of resources and cost.

Impact/Benefit

- Industry Standard Best Practices for Capacity Planning
- Multi-tenancy where applicable.
- Helped team setting up Performance KPIs and Autoscaling parameters.
- Effective use of Pod Affinities and Node Affinities.
- Separate Strategy for Non-Prod Environments and Prod Environments.
- Metrics Best Practices and Observability setup.

Impact and Benefits:

- Able to Reduce 10-15% of AWS Total Cost.
- Improved usage of Resources
- Observability Best Practices to make troubleshooting easier.
- Improved KPI Performances.

Network Cost Optimization

- ***Optimize pod placement and communication***
Service Internal Traffic Policy
- ***Topology Aware Routing***
Topology Aware Hints
Works at Network Layer and not at application layer



Internal Traffic Policy – Pod to Pod

apiVersion: v1

Kind: Service

Metadata:

name: my-service

Spec:

internalTrafficPolicy: Local | Cluster

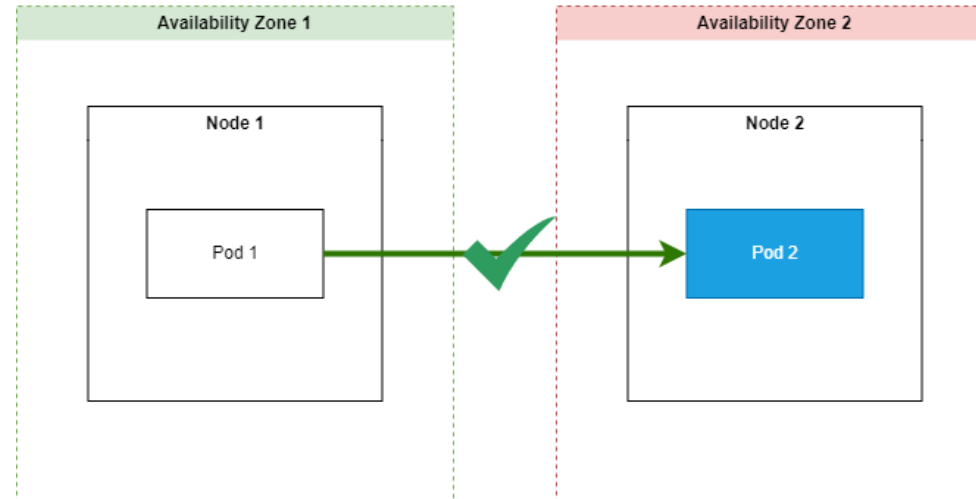


Internal Traffic Policy

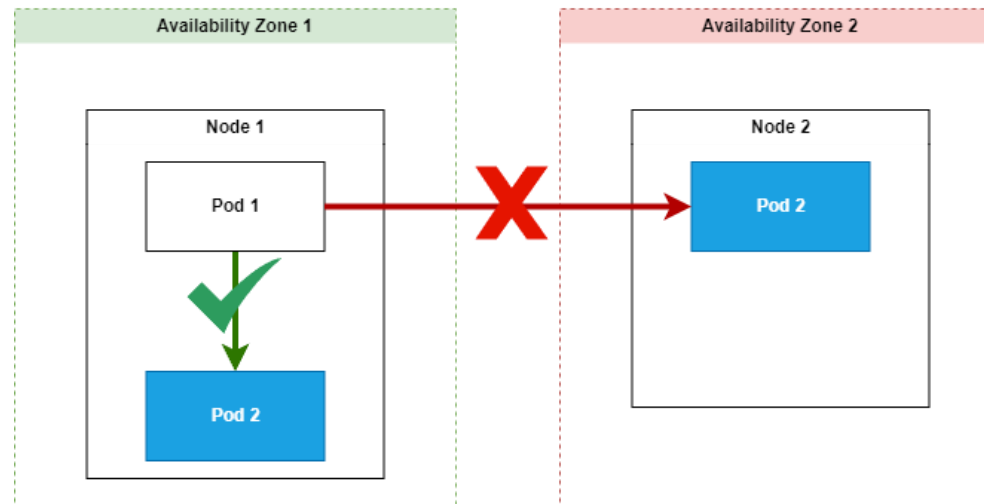
Cluster Traffic Policy: Routes traffic to any available endpoints, regardless of their location in the cluster. Requests will automatically route to other nodes if no local endpoints are available, which can be useful for balancing loads across nodes but can introduce added latency and cost

Local Traffic Policy: Routes traffic only to endpoints on the same node as the originating pod. If no local endpoints exist, the request will fail, providing a low-latency path while requiring careful planning for endpoint availability.

Cluster Traffic Policy



Local Traffic Policy



Topology Aware Routing - Configurations

Kubectll label nodes <node-name> topology.Kubernetes.io/region=us-west-1

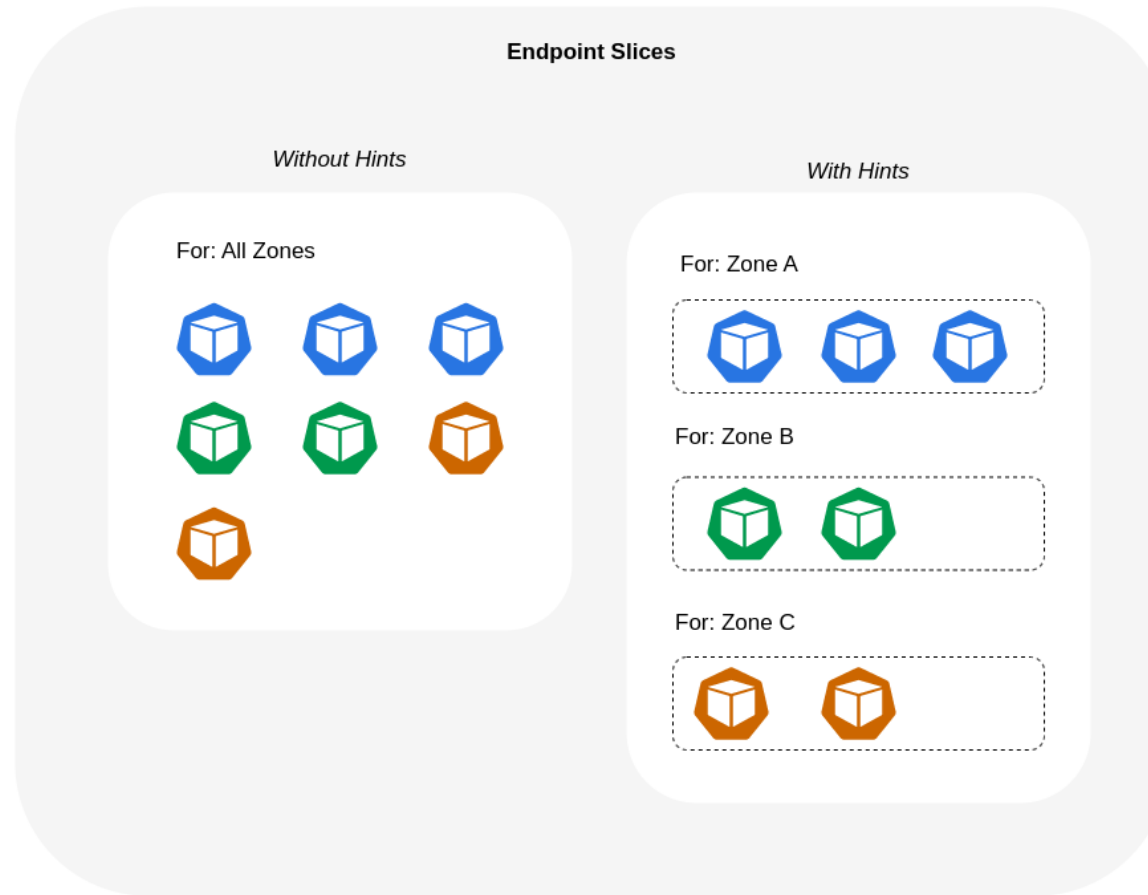
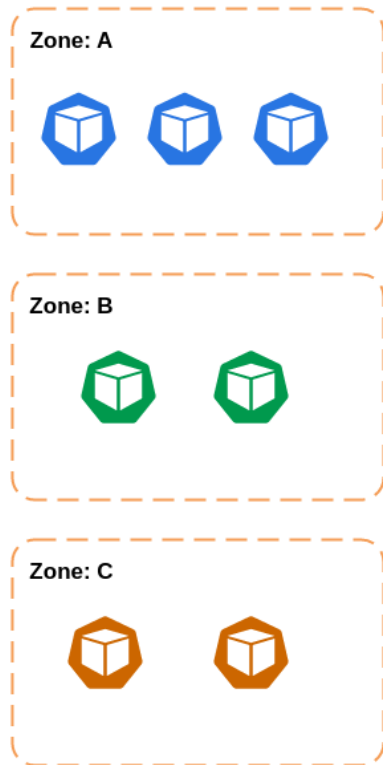
Kubectll label nodes <node-name> topology.Kubernetes.io/region=us-east-1

Kubectll label nodes <node-name> topology.Kubernetes.io/region=europe-west-1

Service.yml

```
apiVersion: v1
Kind: Service
Metadata:
  name: my-service
Spec:
  selector:
    app:my-app
  ports:
    -protocol:TCP
      port:80
      targetport: 9376
  topologyKeys:
    -"topology.Kubernetes.io/zone"
    -"*"
```

EndPoint Slices



1.EndpointSlices with Hints: The EndpointSlice controller assigns hints to each endpoint, specifying the zone it should serve.

2.Traffic Direction by kube-proxy: When traffic is routed through kube-proxy, it uses these hints to ensure that most traffic remains within the local zone unless load balancing requires otherwise.

Things to Keep in Mind

- Ensure Pod Distribution across Zones
 - Example there are 2 Nodes(2 Zones) in a cluster
 - Zone A – 10 % Pods
 - Zone B – 20 % Pods
- Is TAR an ideal solution in this scenario – No
- In this use case Pod Affinity along with TAR will be an optimal solution



Storage Optimization

Use the Right Storage Class and Better to go with Dynamic Provisioning specific to your cloud. Ex gp3 for AWS, Premium SSD/Ultra Disk for Azure and Regional Persistent discs for GCP

Adjust and Set Storage class parameters based on performance and cost. Ex: IOPS, Throughput and Volume Expansion capabilities.

`allowVolumeExpansion: true` - This helps PVC grow automatically and also helps reduces waste storage while sudden disk space issue spikes up

Efficient use of Log Retentions and Life Cycle Management and regular clean up of orphan resources.

Appendix

Pod Placement

Type	Purpose	Governed By	Opposite Mechanism
Node Affinity	Attracts pod to specific nodes	Node Labels	Node Taints(Not Node AntiAffinity)
Pod Affinity	Attracts pod to other pods	Pod Labels + Topology	Pod Anti Affinity
Pod Anti Affinity	Repels pods from other pods	Pod Labels + Topology	Pod Affinity

Topology Aware Routing - Configurations

PodAffinity Example

```
podAffinity:  
  requiredDuringSchedulingIgnoredDuringExecution:  
    -labelSelector:  
      matchExpressions:  
        -key: app  
          operator: In  
          values: [ui]  
      topologyKey: "topology.Kubernetes.io/zone"
```

Explanation

- Schedule this pod in the same zone where a pod labelled **ui** and running
- If nodes are labelled with topology.Kubernetes.io/zone=us-east1-1b, then scheduler ensures the new pod lands in the same zone as the **ui** pod.

TAR Key Benefits and Best Practices

Key Benefits

- 1.Reduced Cross-Zone Traffic Costs:** Routing traffic within the same zone eliminates cross-zone data transfer costs, a significant advantage in multi-AZ environments.
- 2.Lower Latency:** By keeping traffic local, the latency involved in cross-zone communication is minimized, enhancing the performance of latency-sensitive applications.
- 3.Better Resource Allocation:** This routing strategy also ensures efficient use of network resources by minimizing unnecessary data hops.

Best Practices

- 1.Even Workload Distribution:** For the best results, ensure that workloads are evenly distributed across all zones. This helps in maintaining high availability and prevents one zone from being overloaded.
- 2.Pod Topology Spread Constraints:** Use PodTopologySpreadConstraints to distribute pods evenly across zones, which works in tandem with topology-aware routing to ensure efficiency and fault tolerance.
- 3.Node Affinity and Pod Affinity:** To further control traffic locality, consider using node affinity or pod affinity to place related workloads on nodes that will benefit from lower-latency communication.

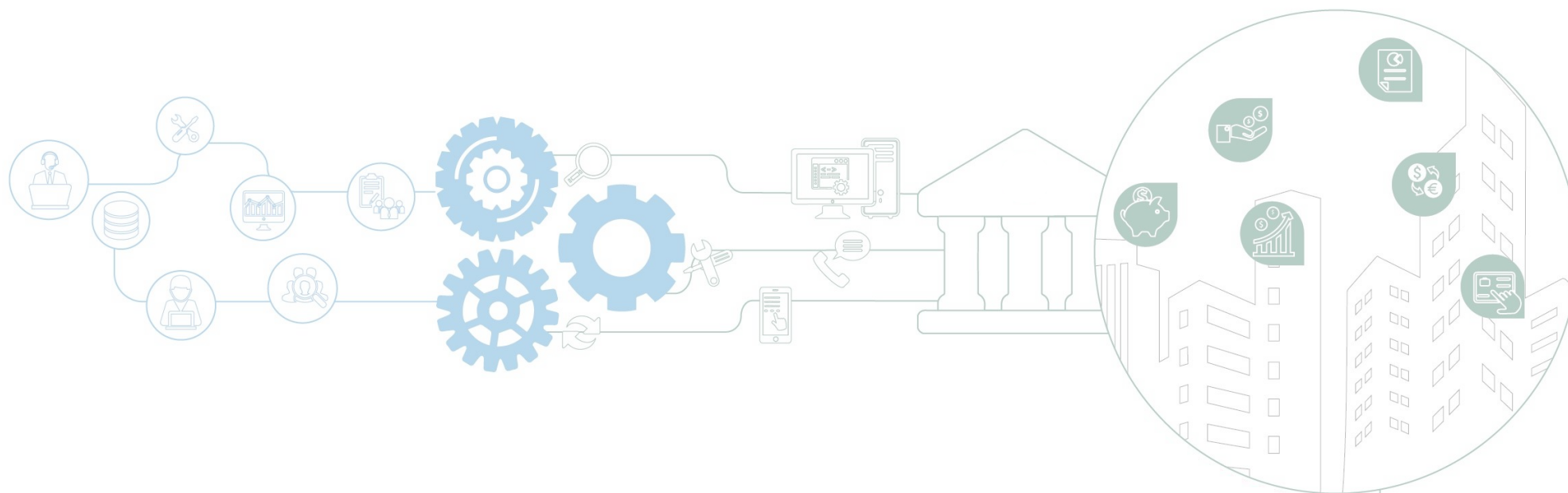
TAR Limitations and Use cases

Limitations

- Service Internal Traffic Policy:** Topology-aware routing is not compatible with the Local internal traffic policy, which also aims to restrict traffic to a particular node.
- Scaling issues:** If scaling is not optimized for topology zones property, resource allocation might not happen properly making services unavailable in the availability zone with the least resources, and even though HPA is set since traffic is routed based on topology so it will not trigger scale up as resources in other availability zone.

Usecases

- Microservices Architectures:** Where frequent inter-service communication can lead to high costs if services are distributed across multiple zones.
- Latency-Sensitive Applications:** Where keeping network traffic within the local zone is critical for maintaining low latency and high user experience.
- Hybrid Workloads:** Applications deployed across multiple cloud environments or on-premises data centers where cost and performance management are crucial.



THANK YOU